Chat Portal with End-to-End Encryption
and Distributed FTPS File Storage

Submitted By

Muhammad Umar | 23K-0023 (Project Leader)
Full-Stack Chat Architecture & End-to-End Encryption

Fasih Hasan Khan | 23K-0018
Distributed FTPS Storage Layer & File Chunking Subsystem

Instructor: Sir Shoaib Raza

Computer Networks – Final Year Project Report

FAST School of Computing
National University of Computer & Emerging Sciences

Motivation

In an era defined by pervasive digital communication, the confidentiality of personal and professional conversations has become a matter of fundamental importance. Mainstream messaging platforms, while functionally rich, frequently operate under centralised server architectures where message plaintext is accessible to the service provider, rendering users vulnerable to data breaches, insider threats, and government-mandated surveillance. This reality poses a direct conflict with the core networking principle of data confidentiality, which holds that information exchanged between two parties should remain intelligible only to those parties.

This project was motivated by the desire to apply, in a tangible and demonstrable system, the cryptographic and networking concepts studied throughout the Computer Networks curriculum. By constructing a chat portal that enforces End-to-End Encryption (E2EE) at the browser layer, alongside a distributed File Transfer Protocol Secure (FTPS) storage backend, the team sought to demonstrate that zero-knowledge server architecture — where the server routes and stores data it fundamentally cannot read — is achievable using standard, well-understood protocols: RSA-OAEP, AES-GCM, WebSockets, and TLS-protected FTP.

Furthermore, the academic value of the project lies in the integration of multiple distinct networking layers into a single coherent system: a real-time WebSocket relay, a RESTful HTTP API, a relational persistence layer, and a multi-server TLS file-storage subsystem. Each of these components represents a practical application of theory, making the project both a meaningful engineering artefact and a comprehensive demonstration of acquired competence in network application design.

Overview

Significance of the Project

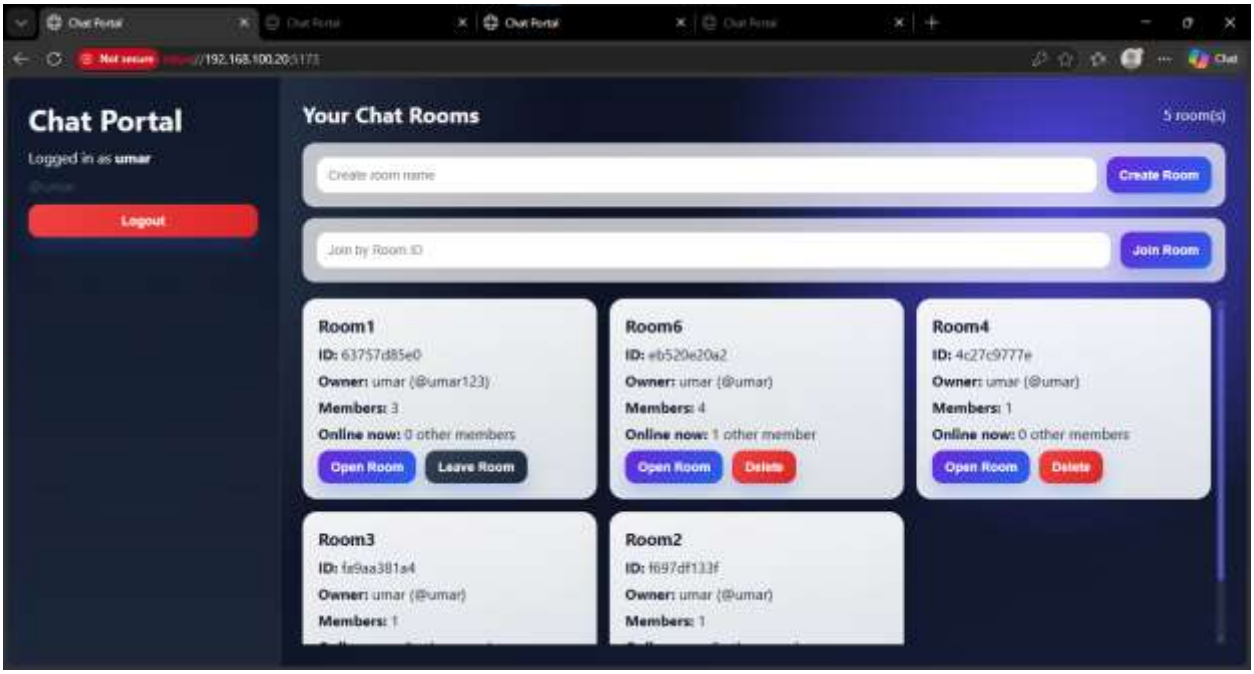
The Chat Portal with E2EE and Distributed FTPS File Storage addresses a well-established gap in educational software: most computer science projects that involve networking focus either on the transport layer or the application layer in isolation. This project demonstrates a vertically integrated system that spans all layers simultaneously — from raw TLS socket management on the FTPS servers, through HTTP/WebSocket application-layer protocols on the FastAPI backend, to browser-native cryptographic operations at the user interface layer.

The practical significance of this architecture is that it serves as a proof of concept for the "zero-knowledge server" model. The server operator, even with full database access and packet capture capability, cannot reconstruct any message or file in plaintext. All sensitive data is encrypted client-side before transmission and decrypted client-side after receipt. This model is the gold standard employed by modern secure messaging applications such as Signal and WhatsApp, and its successful implementation within an academic context demonstrates mastery of both network security theory and applied software engineering.

Description of the Project

The system comprises three major subsystems working in concert:

- React Frontend: A single-page application (SPA) built with React and bundled by Vite. The frontend is solely responsible for all cryptographic operations. It generates RSA key pairs using the browser's native Web Crypto API, encrypts outgoing messages and files with AES-GCM before they leave the browser, and decrypts incoming ciphertext using the user's locally-stored RSA private key. The private key is never transmitted to any server under any circumstance.
- FastAPI Backend: An asynchronous Python server exposing REST endpoints for authentication and room management, a WebSocket endpoint for real-time message relay, and HTTP endpoints for encrypted file upload and download. The backend persists all data — users, rooms, messages, file metadata, and file chunk locations — in a local SQLite database. Crucially, the backend handles only opaque, encrypted byte sequences; it has no mechanism to decrypt them.
- Distributed FTPS Storage Layer: Three independent TLS-enabled FTP servers, each running on a distinct port (2121, 2122, 2123) with separate storage directories. The FastAPI backend distributes encrypted file chunks across these three servers using a deterministic round-robin algorithm, storing chunk-to-server mapping metadata in SQLite for reassembly on download.



Background of the Project

The conceptual foundations of this project draw from several established areas of computer science and network security.

End-to-End Encryption (E2EE)

E2EE is a communication paradigm in which only the communicating users can read the messages. It prevents potential eavesdroppers — including telecom providers, internet providers, and the provider of the communication service — from being able to access the cryptographic keys needed to decrypt the conversation [1]. The Signal Protocol, introduced by Open Whisper Systems, popularised the concept of E2EE in consumer messaging. This project implements a simplified variant of the hybrid encryption model underlying Signal: asymmetric RSA for key encapsulation and symmetric AES for bulk data encryption.

Hybrid Encryption: RSA-OAEP and AES-GCM

The Web Crypto API specification [2] defines `RSA-OAEP` (Optimal Asymmetric Encryption Padding with SHA-256) as the standard algorithm for public-key encryption of small payloads, and `AES-GCM` (Advanced Encryption Standard in Galois/Counter Mode with a 256-bit key) for authenticated symmetric encryption. This project uses this exact pairing: a fresh 256-bit AES-GCM key is generated for each message, the message payload is encrypted with that key, and then the key itself is RSA-OAEP encrypted once per recipient using that recipient's public RSA key. This approach solves the fundamental inefficiency of pure asymmetric encryption while retaining strong confidentiality guarantees [3].

WebSocket Protocol

The WebSocket protocol (RFC 6455) provides full-duplex communication over a single TCP connection. Unlike long-polling HTTP, WebSocket connections remain open indefinitely, enabling the server to push data to clients instantaneously. This project uses FastAPI's native WebSocket support to relay encrypted message packets to all members of a chat room in real time, maintaining room presence state through heartbeat pings and online-count broadcasts [4].

FTP over TLS (FTPS)

FTPS is defined in RFC 4217 as an extension to the File Transfer Protocol (FTP) that adds support for TLS. Unlike SFTP (which is a completely different protocol based on SSH), FTPS is a direct extension of FTP that negotiates a TLS session on the control channel and optionally on the data channel. The project uses the pyftpdlib library [5] to run three independent TLS-enabled FTP servers, each serving as a shard of the distributed file storage layer. Python's built-in ftplib.FTP_TLS class is used by the backend to communicate with these servers over TLS-protected connections using PROT P (private data protection) mode.

Chunked File Storage

Distributing large files across multiple storage nodes by splitting them into fixed-size chunks is a technique foundational to distributed file systems such as Google File System (GFS) and Hadoop

Distributed File System (HDFS) [6]. This project applies a simplified version of this concept: the FastAPI backend splits the encrypted file payload into 1 MiB chunks and distributes them across three FTPS servers using round-robin placement. Chunk metadata — specifically the server ID and remote path for each chunk index — is stored in SQLite, enabling the backend to reassemble the complete file in the correct order upon download.

Project Category

This project is a Product-Based project. The deliverable is a fully functional, locally deployable, end-to-end encrypted chat and file-sharing application. The project does not propose a novel research hypothesis, nor is it commissioned by an external industry partner; rather, it constructs a working software product that applies well-established network security principles to produce a demonstrably functional system.

The product encompasses a full software stack — frontend, backend, and storage layer — and is designed to run entirely on a single development machine for demonstration and evaluation purposes. All components are open-source and free to deploy, making the product accessible without commercial licensing costs.

Features / Scope / Modules

1. End-to-End Encrypted Messaging

The most significant feature of the system. Every message is encrypted in the sender's browser using AES-GCM with a per-message 256-bit key before transmission. The AES key is individually RSA-OAEP wrapped for every room member using that member's public key. The backend stores and relays only the opaque ciphertext packet; no server-side entity — present or future — can decrypt messages without access to the recipient's RSA private key, which never leaves the browser.

2. Distributed FTPS File Storage with Client-Side Encryption

Files are encrypted in the browser before upload, making the server storage layer a zero-knowledge encrypted archive. The FastAPI backend splits the encrypted payload into 1 MiB chunks using a configurable FTPS_CHUNK_SIZE_BYTES parameter and distributes them across three independent TLS-enabled FTP servers using deterministic round-robin placement. Each chunk is stored at a structured remote path (room_<id>/file_<id>/chunk_<n>.bin), and chunk-to-server metadata is persisted in SQLite. On download, the backend reassembles the chunks and streams the encrypted payload back to the browser, which decrypts the file locally and triggers a client-side save.

3. Real-Time WebSocket Communication with Presence Tracking

Room communication uses the WebSocket protocol for bidirectional, low-latency message delivery. The backend maintains a map of active WebSocket connections per room and broadcasts incoming encrypted packets to all connected members. User presence is tracked

through a heartbeat mechanism: the frontend sends a POST /auth/ping request every 25 seconds, updating the user's last_seen_at timestamp. The room's live online count is computed using a configurable sliding time window and broadcast to all room members whenever a user connects or disconnects.

4. RSA Key Pair Generation and Public Key Infrastructure

On first use, each browser generates a 2048-bit RSA key pair using the Web Crypto API (with a fallback to node-forge for environments where window.crypto.subtle is unavailable). The private key is stored in the browser's localStorage and never transmitted to any server. The public key is exported in SPKI format, base64-encoded, and uploaded to the backend via PUT /auth/public-key, where it is stored in the users table and relayed to room members in real time through WebSocket announce packets. This mechanism provides a lightweight, self-bootstrapping public key infrastructure without a certificate authority.

5. Comprehensive Room and Membership Management

The system supports full room lifecycle management. Any authenticated user may create a room, becoming its owner, or join an existing room by its unique ID. Room owners have exclusive authority to remove members and delete the room entirely. Ownership may be transferred to another member when the owner chooses to leave. Soft-delete is implemented for messages: the sender may delete their own messages, causing the backend to mark them with is_deleted = 1 and a deleted_at timestamp; clients render such messages as "This message was deleted" rather than removing the record, preserving conversation continuity.

6. Crypto Fallback and Key Continuity Handling

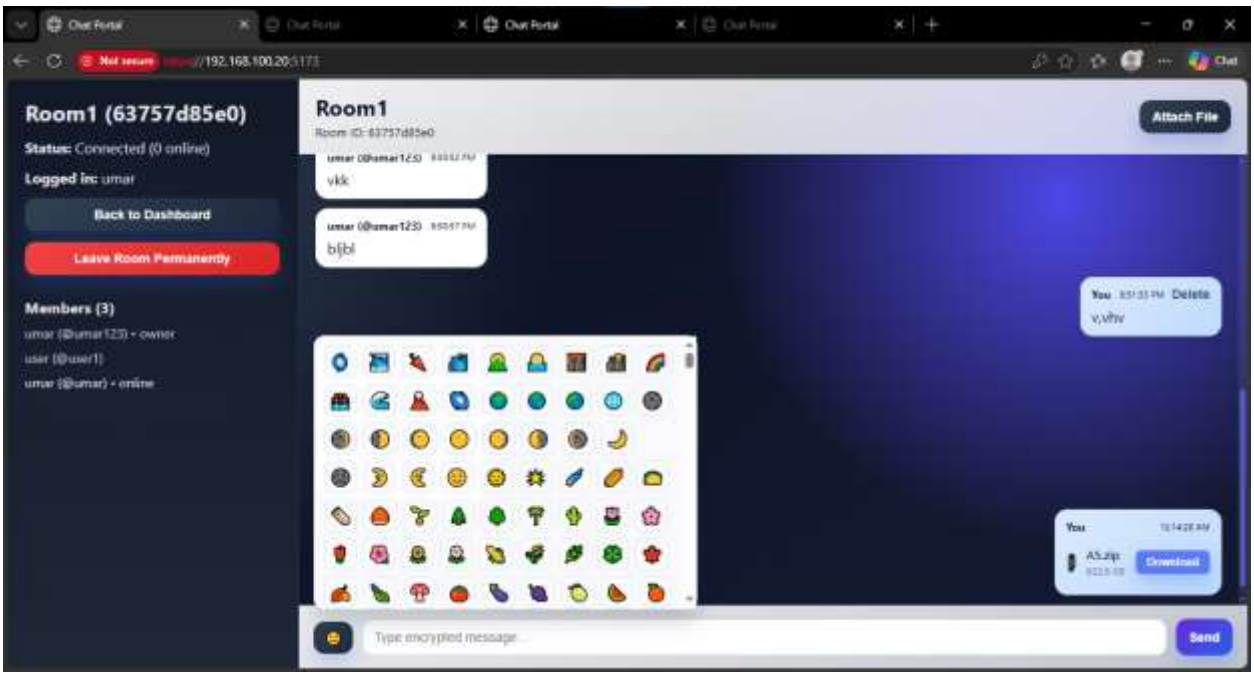
The browser crypto helper in crypto.js implements a two-path design: the primary path uses the native Web Crypto API, while the fallback path uses the node-forge library for environments where window.crypto.subtle is unavailable (e.g., insecure HTTP origins or older browsers). Both paths produce and consume identical ciphertext formats, ensuring interoperability. The UI explicitly detects and communicates scenarios where the local private key is absent or where an incoming message's encryptedKeys map does not contain an entry for the local user, presenting a clear, actionable error instead of silently failing to display a message.

7. WebSocket Automatic Reconnection Logic

The frontend WebSocket client distinguishes between network-level disconnects (which it attempts to recover from automatically) and server-initiated closures with specific close codes indicating permission or membership changes (e.g., being removed from a room or the room being deleted). For the latter class, reconnection is suppressed to prevent the client from repeatedly attempting to rejoin a room it has been explicitly ejected from.

8. Emoji Picker and Rich UI Chat Composer

The message composer includes an integrated emoji picker, supporting expressive communication without requiring external services. The React frontend provides a responsive layout that displays room lists with live member counts and online presence indicators, a scrollable message history with per-message sender identification and timestamps, and in-chat system notifications for join, leave, ownership-transfer, and room-deletion events.



Project Planning

The project was executed over a ten-week development schedule, divided between the two team members according to their primary areas of responsibility. Muhammad Umar (23K-0023) served as Project Leader and was responsible for the full-stack chat architecture, the FastAPI REST and WebSocket backend, the SQLite database schema, the React frontend, and the end-to-end encryption subsystem. Fasih Hasan Khan (23K-0018) was responsible for the design and implementation of the distributed FTPS storage layer, the file chunking and reassembly logic, and the client-side file encryption and decryption workflow.

Table 1 below presents the project schedule in the form of a Gantt-style milestone matrix, indicating the weeks during which each milestone was active.

Milestone / Task	Week 1–2	Week 3–4	Week 5–6	Week 7–8	Week 9–10
Requirements & Design	✓				
Backend: Auth, Rooms, DB	✓	✓			

Milestone / Task	Week 1–2	Week 3–4	Week 5–6	Week 7–8	Week 9–10
E2EE Crypto Layer (Umar)		✓	✓		
WebSocket & Real-time Chat			✓	✓	
FTPS Layer & Chunking (Fasih)		✓	✓	✓	
Frontend UI & Integration			✓	✓	
Testing & Debugging				✓	✓
Report & Documentation					✓

Table 1. Project Milestone Schedule

Each major milestone involved iterative development and testing cycles. The requirements and design phase established the data flow architecture, the SQLite schema, the WebSocket message packet format, and the cryptographic protocol decisions. The backend was developed incrementally, beginning with authentication and room management before layering in WebSocket relay and file storage. The E2EE subsystem and the FTPS layer were developed in parallel and integrated during the fifth and sixth weeks, with a dedicated testing phase to validate correct ciphertext round-tripping across both the message and file pathways.

Project Feasibility

Technical Feasibility

The project is fully realised using mature, well-documented, and freely available technologies. Python's FastAPI is a production-grade ASGI framework with native WebSocket support and extensive documentation [7]. The Web Crypto API is implemented in every major browser (Chrome, Firefox, Safari, Edge) and the node-forge library provides a reliable fallback. SQLite is a stable, zero-configuration embedded database engine that is part of the Python standard library. The pyftpdlib library provides production-quality FTP server functionality with TLS support and has been in active maintenance since 2007 [5].

The primary technical risk involved ensuring that the binary ciphertext produced by the Web Crypto API AES-GCM implementation and the node-forge fallback implementation were format-compatible, such that a message encrypted on a browser supporting Web Crypto could be decrypted on a browser relying on node-forge. This risk was mitigated through a shared encoding convention (base64 serialisation of IV and ciphertext) and comprehensive integration testing across both paths.

A secondary technical consideration was the management of three concurrent FTPS server processes alongside the FastAPI backend process. This was addressed by launching all FTPS servers in background threads from a single ftp_servers.py entry point, sharing the same Python process address space as the FastAPI application for simplified deployment.

Economic Feasibility

The entire software stack is open-source and free to use. No commercial software licences, cloud services, or paid APIs are required at any stage of development or deployment. The system is designed to run entirely on a single development machine, eliminating hosting costs for demonstration purposes. The hardware requirements — a standard laptop or desktop with a modern CPU and 8 GB of RAM — are met by the equipment already available to the development team through the university's computing resources.

The development cost is purely in terms of time invested by the two team members, estimated at approximately 60–80 person-hours of design, development, testing, and documentation work over the ten-week project period.

Schedule Feasibility

The ten-week development schedule was sufficient to produce a complete, demonstrable system with all planned features implemented. The parallel development model — with Muhammad Umar focusing on the chat and encryption subsystem and Fasih Hasan Khan focusing on the FTPS storage subsystem — allowed both major technical challenges to be addressed simultaneously, reducing the critical path length. The final two weeks allocated to testing and documentation provided adequate time to identify and resolve integration issues and to prepare this report.

Hardware and Software Requirements

Table 2 below provides a comprehensive inventory of all hardware and software dependencies required to develop, run, and demonstrate the system.

Category	Component	Specification / Purpose
Hardware	Development Machine	Dual-core CPU, 8 GB RAM, 20 GB disk space minimum
Hardware	Network Interface	Standard Ethernet / Wi-Fi for localhost loopback testing
Software – Runtime	Python 3.10+	Backend execution environment for FastAPI
Software – Runtime	Node.js 18+	Frontend build toolchain (Vite + React)
Software – Framework	FastAPI	Async Python web framework for REST & WebSocket APIs
Software – Framework	React 18 + Vite	Single-page frontend application with hot-reload
Software – Storage	SQLite 3	Embedded relational database for metadata persistence
Software – Storage	pyftpdlib 1.5+	Pure-Python FTPS server library (three instances)

Category	Component	Specification / Purpose
Software – Crypto	Web Crypto API	Browser-native hardware-accelerated cryptographic primitives
Software – Crypto	node-forge	JavaScript fallback library for RSA-OAEP / AES-GCM
Software – Security	Self-signed TLS cert	TLS 1.2+ for FTPS channel encryption on all three servers
Dev Tools	VS Code / PyCharm	IDE for development and debugging
Dev Tools	Git	Version control and collaboration

Table 2. Hardware and Software Requirements

All software dependencies are installable via standard package managers (pip for Python packages, npm for Node.js packages) with no manual compilation required. The self-signed TLS certificate used by the FTPS servers is generated using OpenSSL and bundled with the repository. No external certificate authority or domain name is required for local deployment.

```
(.venv) PS D:\Code\CN\Project\backend> pip install -r requirements.txt
Requirement already satisfied: fastapi==0.115.12 in d:\code\cn\project\backend\.venv\lib\site-packages (from -r requirement
s.txt (line 1)) (0.115.12)
Requirement already satisfied: uvicorn==0.34.0 in d:\code\cn\project\backend\.venv\lib\site-packages (from uvicorn[standard
]==0.34.0->-r requirements.txt (line 2)) (0.34.0)
Requirement already satisfied: pyftpdlib==2.0.1 in d:\code\cn\project\backend\.venv\lib\site-packages (from -r requirements
.txt (line 3)) (2.0.1)
Requirement already satisfied: pyOpenSSL==24.3.0 in d:\code\cn\project\backend\.venv\lib\site-packages (from -r requirement
s.txt (line 4)) (24.3.0)
Requirement already satisfied: python-multipart==0.0.12 in d:\code\cn\project\backend\.venv\lib\site-packages (from -r requ
irements.txt (line 5)) (0.0.12)
Requirement already satisfied: starlette<0.47.0,>=0.40.0 in d:\code\cn\project\backend\.venv\lib\site-packages (from fastap
i==0.115.12->-r requirements.txt (line 1)) (0.46.2)
Requirement already satisfied: pydantic!=1.8,!=1.8.1,!=2.0.0,!=2.0.1,!=2.1.0,<3.0.0,>=1.7.4 in d:\code\cn\project\backend\.
venv\lib\site-packages (from fastapi==0.115.12->-r requirements.txt (line 1)) (2.13.4)
Requirement already satisfied: typing-extensions>=4.8.0 in d:\code\cn\project\backend\.venv\lib\site-packages (from fastapi
==0.115.12->-r requirements.txt (line 1)) (4.15.0)
Requirement already satisfied: click>=7.0 in d:\code\cn\project\backend\.venv\lib\site-packages (from uvicorn==0.34.0->uvic
orn[standard]==0.34.0->-r requirements.txt (line 2)) (8.3.3)
Requirement already satisfied: h11>=0.8 in d:\code\cn\project\backend\.venv\lib\site-packages (from uvicorn==0.34.0->uvicor
n[standard]==0.34.0->-r requirements.txt (line 2)) (0.16.0)
Requirement already satisfied: pyasyncore in d:\code\cn\project\backend\.venv\lib\site-packages (from pyftpdlib==2.0.1->-r
requirements.txt (line 3)) (1.0.5)
Requirement already satisfied: pyasynchat in d:\code\cn\project\backend\.venv\lib\site-packages (from pyftpdlib==2.0.1->-r
requirements.txt (line 3)) (1.0.5)
Requirement already satisfied: cryptography<45,>=41.0.5 in d:\code\cn\project\backend\.venv\lib\site-packages (from pyOpenS
SL==24.3.0->-r requirements.txt (line 4)) (44.0.3)
Requirement already satisfied: colorama>=0.4 in d:\code\cn\project\backend\.venv\lib\site-packages (from uvicorn[standard]=
=0.34.0->-r requirements.txt (line 2)) (0.4.6)
Requirement already satisfied: httptools>=0.6.3 in d:\code\cn\project\backend\.venv\lib\site-packages (from uvicorn[standar
d]==0.34.0->-r requirements.txt (line 2)) (0.7.1)
Requirement already satisfied: python-dotenv>=0.13 in d:\code\cn\project\backend\.venv\lib\site-packages (from uvicorn[stan
dard]==0.34.0->-r requirements.txt (line 2)) (1.2.2)
Requirement already satisfied: pyyaml>=5.1 in d:\code\cn\project\backend\.venv\lib\site-packages (from uvicorn[standard]==0
.34.0->-r requirements.txt (line 2)) (6.0.3)
Requirement already satisfied: watchfiles>=0.13 in d:\code\cn\project\backend\.venv\lib\site-packages (from uvicorn[standar
```

```
(.venv) PS D:\Code\CN\Project> cd .\frontend\
(.venv) PS D:\Code\CN\Project\frontend> npm install

  up to date, audited 65 packages in 10s

  7 packages are looking for funding
    run `npm fund` for details

  2 moderate severity vulnerabilities

  To address all issues (including breaking changes), run:
    npm audit fix --force

  Run `npm audit` for details.
(.venv) PS D:\Code\CN\Project\frontend>
```

Diagrammatic Representation of the Overall System

The figure below presents the complete end-to-end system architecture, illustrating the data flow from the browser's cryptographic operations through the FastAPI relay layer to the distributed FTPS storage subsystem. The diagram was produced using mermaid.live and depicts the key interactions between all system components: the React SPA, the FastAPI server, the SQLite database, and the three TLS-protected FTPS servers.

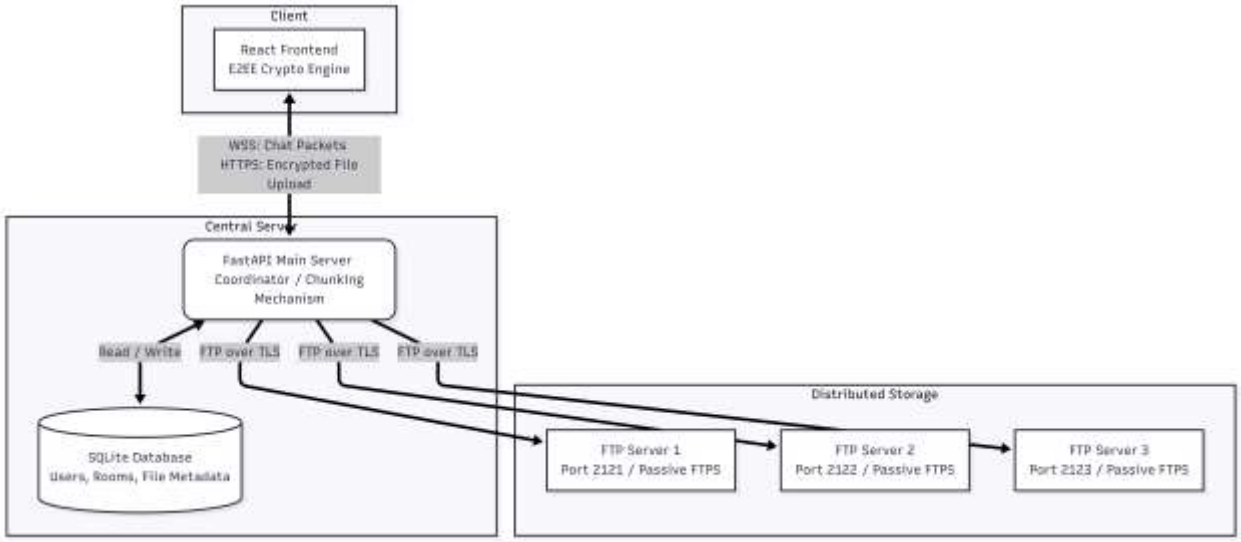


Figure 1. End-to-End System Architecture — Chat Portal with Distributed FTPS Storage

The architecture enforces a strict principle of data confidentiality: at no point in the message or file path does unencrypted user content traverse the server or the FTPS storage nodes. The arrows on the left side of the diagram (browser-to-server) carry only AES-GCM ciphertext and RSA-wrapped key bundles. The arrows on the right side (server-to-FTPS) carry only the encrypted file chunks produced by the browser-side AES-GCM encryption step. The SQLite database stores only ciphertext, metadata, and public keys — never private keys or plaintext.

References

[1] M. Marlinspike and T. Perrin, *The Signal Protocol: A Non-Federated Ratcheting Forward Secrecy Protocol*, Open Whisper Systems, 2016. [Online]. Available: https://signal.org/docs/

[2] W3C, *Web Cryptography API*, W3C Recommendation, Jan. 2017. [Online]. Available: https://www.w3.org/TR/WebCryptoAPI/

[3] A. J. Menezes, P. C. van Oorschot, and S. A. Vanstone, *Handbook of Applied Cryptography*, CRC Press, Boca Raton, FL, 1996, ch. 8: "Public-Key Encryption".

[4] I. Fette and A. Melnikov, *The WebSocket Protocol*, IETF RFC 6455, Dec. 2011. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc6455

[5] G. Rodola, *pyftpdlib — Python FTP Server Library*, 2007–2024. [Online]. Available: https://github.com/giampaolo/pyftpdlib

[6] S. Ghemawat, H. Gobioff, and S.-T. Leung, "The Google File System," in *Proc. 19th ACM Symposium on Operating Systems Principles (SOSP)*, Bolton Landing, NY, Oct. 2003, pp. 29–43.

[7] S. Ramírez, *FastAPI — Modern, Fast Web Framework for Python*, 2018–2024. [Online]. Available: https://fastapi.tiangolo.com

[8] P. Ford-Hutchinson, "Securing FTP with TLS," IETF RFC 4217, Oct. 2005. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc4217